

LAPORAN PRAKTIKUM 9



Nama : Akmallutfi Afza Gusty

NIM : 707012500064

Kelas : D4 SIKC 49-04

Mata kuliah : Pemrograman Web

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep HMVC pada CodeIgniter 3.
2. Mengimplementasikan struktur modular (feature-based).
3. Membuat REST API berbasis JSON.
4. Mengintegrasikan Model dengan REST Controller.
5. Menggunakan AJAX untuk komunikasi asynchronous.
6. Membuat fitur CRUD tanpa refresh halaman.
7. Menguji API menggunakan Postman.
8. Mengintegrasikan frontend dengan REST API

Untuk melaksanakan praktikum ini, diperlukan:

1. XAMPP (Apache & MySQL aktif)
2. PHP 7.x
3. CodeIgniter 3
4. Library HMVC (Modular Extensions – MX)
5. Library REST Server (chriskacerguis)
6. Postman
7. VS Code / Code Editor
8. Browser (Chrome/Firefox)

Langkah-Langkah Praktikum

API

```
<?php
// Mencegah file controller diakses langsung tanpa
lewat CodeIgniter.
defined('BASEPATH') OR exit('No direct script access
allowed');

// Controller API untuk fitur autentikasi user seperti
register, login, logout, dan data user login.
```

```
class Auth extends MX_Controller {

    // Constructor dijalankan otomatis saat controller
    dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan MX_Controller.
        parent::__construct();
        // Memuat model User_model dari module Api
        dengan alias User_model.
        $this->load->model('Api/User_model',
        'User_model');
        // Memuat library JWT untuk membuat dan
        memverifikasi token.
        $this->load->library('jwt');
        // Memuat library form_validation untuk
        kebutuhan validasi input.
        $this->load->library('form_validation');
    }

    // Endpoint API untuk mendaftarkan user baru.
    public function register()
    {
        // Mengatur response menjadi JSON.
        $this->_json();

        // Menjalankan proses register dalam blok try
        agar error bisa ditangkap.
        try {
            // Mengambil input request dari JSON body
            atau form POST.
            $input = $this->_request_input();

            // Mengecek apakah field name, email, dan
            password kosong.
            if (empty($input['name']) ||
            empty($input['email']) || empty($input['password'])) {
```

```

        // Mengirim response error jika input
wajib belum lengkap.
        return $this->_response(false, 'Name,
email, and password are required', null, 400);
    }

    // Mengecek apakah format email valid.
    if (!filter_var($input['email'],
FILTER_VALIDATE_EMAIL)) {
        // Mengirim response error jika format
email salah.
        return $this->_response(false, 'Invalid
email format', null, 400);
    }

    // Mengecek apakah email sudah terdaftar.
    if ($this->User_model->email_exists($input['email'])) {
        // Mengirim response conflict jika
email sudah digunakan.
        return $this->_response(false, 'Email
already registered', null, 409);
    }

    // Mengecek apakah panjang password minimal
6 karakter.
    if (strlen($input['password']) < 6) {
        // Mengirim response error jika
password terlalu pendek.
        return $this->_response(false,
'Password must be at least 6 characters', null, 400);
    }

    // Menyiapkan data user yang akan disimpan
ke database.
    $user_data = array(
        // Membersihkan nama dari karakter
HTML.

```

```

        'name' =>
htmlspecialchars($input['name']),
        // Mengubah email menjadi huruf kecil
agar konsisten.
        'email' => strtolower($input['email']),
        // Mengenkripsi password menggunakan
bcrypt sebelum disimpan.
        'password' =>
password_hash($input['password'], PASSWORD_BCRYPT)
    );

    // Menyimpan user baru melalui model dan
mengambil ID user.
    $user_id = $this->User_model-
>create($user_data);
    // Mengecek apakah proses pembuatan user
gagal.
    if (!$user_id) {
        // Mengirim response error server jika
user gagal dibuat.
        return $this->_response(false, 'Failed
to create user', null, 500);
    }

    // Mengambil kembali data user yang baru
dibuat.
    $user = $this->User_model-
>get_by_id($user_id);
    // Membuat token JWT berisi data dasar
user.
    $token = $this->jwt->create(array(
        // Menyimpan ID user ke payload token.
        'id' => $user->id,
        // Menyimpan nama user ke payload
token.
        'name' => $user->name,
        // Menyimpan email user ke payload
token.

```

```

        'email' => $user->email
    ));

    // Mengirim response sukses register
    beserta data user dan token.
    return $this->_response(true, 'User
    registered successfully', array(
        // ID user yang baru terdaftar.
        'id' => $user->id,
        // Nama user yang baru terdaftar.
        'name' => $user->name,
        // Email user yang baru terdaftar.
        'email' => $user->email,
        // Waktu user dibuat.
        'created_at' => $user->created_at,
        // Token akses untuk autentikasi
    request berikutnya.
        'access_token' => $token,
        // Jenis token yang digunakan pada
    header Authorization.
        'token_type' => 'Bearer'
    ), 201);
    // Menangkap exception jika terjadi error
    server.
    } catch (Exception $e) {
        // Mengirim response error server beserta
    pesan exception.
        return $this->_response(false, 'Server
    error: ' . $e->getMessage(), null, 500);
    }
}

// Endpoint API untuk login user.
public function login()
{
    // Mengatur response menjadi JSON.
    $this->_json();
}

```

```

        // Menjalankan proses login dalam blok try.
        try {
            // Mengambil input request dari JSON body
atau form POST.
            $input = $this->_request_input();

            // Mengecek apakah email dan password sudah
dikirim.
            if (empty($input['email']) ||
empty($input['password'])) {
                // Mengirim response error jika
email/password kosong.
                return $this->_response(false, 'Email
and password are required', null, 400);
            }

            // Mengambil data user berdasarkan email.
            $user = $this->User_model-
>get_by_email($input['email']);
            // Mengecek apakah user tidak ditemukan
atau password salah.
            if (!$user ||
!password_verify($input['password'], $user->password))
{
                // Mengirim response unauthorized jika
kredensial salah.
                return $this->_response(false, 'Invalid
email or password', null, 401);
            }

            // Membuat token JWT untuk user yang
berhasil login.
            $token = $this->jwt->create(array(
                // Menyimpan ID user ke payload token.
                'id' => $user->id,
                // Menyimpan nama user ke payload
token.
                'name' => $user->name,

```

```

        // Menyimpan email user ke payload
token.
        'email' => $user->email
    ));

    // Mengirim response sukses login beserta
token.
    return $this->_response(true, 'Login
successful', array(
        // ID user yang login.
        'id' => $user->id,
        // Nama user yang login.
        'name' => $user->name,
        // Email user yang login.
        'email' => $user->email,
        // Token akses untuk autentikasi API.
        'access_token' => $token,
        // Jenis token untuk header
Authorization.
        'token_type' => 'Bearer'
    ));
    // Menangkap exception jika terjadi error
server.
    } catch (Exception $e) {
        // Mengirim response error server.
        return $this->_response(false, 'Server
error: ' . $e->getMessage(), null, 500);
    }
}

// Endpoint API logout.
public function logout()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengirim response logout berhasil, token
biasanya dihapus di sisi client.

```



```

        return $this->_response(true, 'Logout
successful');
    }

    // Endpoint API untuk mengambil data user yang
    sedang login.
    public function me()
    {
        // Mengatur response menjadi JSON.
        $this->_json();
        // Memverifikasi token dan mengambil payload
        user.
        $payload = $this->_auth_payload();
        // Jika token tidak valid, proses dihentikan
        karena response error sudah dikirim.
        if (!$payload) {
            return;
        }

        // Mengambil data user berdasarkan ID dari
        payload token.
        $user = $this->User_model->get_by_id($payload-
        >id);
        // Mengecek apakah user masih ada di database.
        if (!$user) {
            // Mengirim response error jika user tidak
            ditemukan.
            return $this->_response(false, 'User not
            found', null, 404);
        }

        // Mengirim response sukses berisi detail user.
        return $this->_response(true, 'User detail',
        array(
            // ID user.
            'id' => $user->id,
            // Nama user.
            'name' => $user->name,

```

```

        // Email user.
        'email' => $user->email,
        // Waktu user dibuat.
        'created_at' => $user->created_at
    ));
}

// Method private untuk mengambil dan memverifikasi
token JWT dari request.
private function _auth_payload()
{
    // Mengambil token dari header Authorization
request.
    $token = $this->jwt->get_token_from_request();
    // Memverifikasi token dan mengambil payload-
nya.
    $payload = $this->jwt->verify($token);
    // Mengecek apakah token tidak valid atau
kosong.
    if (!$payload) {
        // Mengirim response unauthorized jika
token gagal diverifikasi.
        $this->_response(false, 'Unauthorized or
invalid token', null, 401);
        // Mengembalikan false sebagai penanda
autentikasi gagal.
        return false;
    }
    // Mengembalikan payload token jika valid.
    return $payload;
}

// Method private untuk mengatur response sebagai
JSON.
private function _json()
{
    // Membersihkan output buffer agar JSON tidak
tercampur output lain.

```

```

        ob_clean();
        // Mengatur header response menjadi JSON UTF-8.
        header('Content-Type: application/json;
charset=utf-8');
    }

    // Method private untuk mengambil input request.
    private function _request_input()
    {
        // Membaca request body JSON lalu mengubahnya
        menjadi array.
        $input =
        json_decode(file_get_contents('php://input'), true);
        // Jika body bukan JSON valid, gunakan data
        form POST.
        if (!is_array($input)) {
            // Mengambil data dari request POST
            CodeIgniter.
            $input = $this->input->post();
        }
        // Mengembalikan input jika array, jika bukan
        array mengembalikan array kosong.
        return is_array($input) ? $input : array();
    }

    // Method private untuk mengirim response JSON
    standar.
    private function _response($status, $message, $data
    = null, $code = 200)
    {
        // Mengatur HTTP status code response.
        http_response_code($code);
        // Membuat struktur dasar response.
        $response = array('status' => $status,
        'message' => $message);
        // Mengecek apakah ada data tambahan yang perlu
        dikirim.
        if ($data !== null) {

```

```

        // Menambahkan data ke response.
        $response['data'] = $data;
    }
    // Mencetak response dalam format JSON.
    echo json_encode($response,
JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    // Menghentikan eksekusi agar tidak ada output
    tambahan.
    exit;
}
}

```

Post

```

<?php
// Mencegah file controller diakses langsung tanpa
lewat CodeIgniter.
defined('BASEPATH') OR exit('No direct script access
allowed');

// Controller API untuk mengelola data post dengan
method GET, POST, PUT, dan DELETE.
class Post extends MX_Controller {

    // Constructor dijalankan otomatis saat controller
dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan MX_Controller.
        parent::__construct();
        // Memuat model Post_model untuk akses database
post.
        $this->load->model('Post_model');
        // Memuat library JWT untuk validasi token API.
        $this->load->library('jwt');
    }
}

```

```

    // Method utama untuk mengarahkan request API
    berdasarkan HTTP method dan ID.
    public function handle($id = null)
    {
        // Mengambil HTTP method request lalu
        mengubahnya menjadi huruf besar.
        $method = strtoupper($this->input-
        >method(TRUE));

        // Jika method GET tanpa ID, arahkan ke index
        untuk mengambil semua post.
        if ($method === 'GET' && $id === null) {
            // Memanggil method index.
            return $this->index();
        }
        // Jika method GET dengan ID, arahkan ke show
        untuk mengambil satu post.
        if ($method === 'GET' && $id !== null) {
            // Memanggil method show berdasarkan ID.
            return $this->show($id);
        }
        // Jika method POST, arahkan ke create untuk
        membuat post baru.
        if ($method === 'POST') {
            // Memanggil method create.
            return $this->create();
        }
        // Jika method PUT dengan ID, arahkan ke update
        untuk mengubah post.
        if ($method === 'PUT' && $id !== null) {
            // Memanggil method update berdasarkan ID.
            return $this->update($id);
        }
        // Jika method DELETE dengan ID, arahkan ke
        delete untuk menghapus post.
        if ($method === 'DELETE' && $id !== null) {
            // Memanggil method delete berdasarkan ID.
            return $this->delete($id);
        }
    }

```

```

    }

    // Mengirim response jika kombinasi method dan
    URL tidak diizinkan.
    return $this->_response(false, 'Method not
allowed', null, 405);
}

// Endpoint untuk mengambil semua data post.
public function index()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengecek token JWT sebelum mengakses data.
    if (!$this->_auth_payload()) return;

    // Mengambil semua data post dari model.
    $posts = $this->Post_model->get_all();
    // Mengirim response sukses berisi semua post.
    return $this->_response(true, 'Posts retrieved
successfully', $posts);
}

// Endpoint untuk mengambil satu data post
berdasarkan ID.
public function show($id)
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengecek token JWT sebelum mengakses data.
    if (!$this->_auth_payload()) return;

    // Mengambil data post berdasarkan ID.
    $post = $this->Post_model->get_by_id($id);
    // Mengecek apakah post tidak ditemukan.
    if (!$post) {
        // Mengirim response error jika post tidak
ada.

```

```

        return $this->_response(false, 'Post not
found', null, 404);
    }

    // Mengirim response sukses berisi detail post.
    return $this->_response(true, 'Post retrieved
successfully', $post);
}

// Endpoint untuk membuat data post baru.
public function create()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengecek token JWT sebelum membuat data.
    if (!$this->_auth_payload()) return;

    // Membaca request body JSON lalu mengubahnya
menjadi array.
    $input =
json_decode(file_get_contents('php://input'), true);
    // Mengecek apakah field wajib belum lengkap.
    if (empty($input['title']) ||
empty($input['author']) || empty($input['article'])) {
        // Mengirim response error validasi.
        return $this->_response(false, 'Title,
author, and article are required', null, 400);
    }

    // Menyimpan post baru ke database melalui
model.
    $post_id = $this->Post_model->create(array(
        // Mengisi judul post dari input JSON.
        'title' => $input['title'],
        // Mengisi nama author dari input JSON.
        'author' => $input['author'],
        // Mengisi artikel dari input JSON.
        'article' => $input['article']
    ));
}

```

```

    ));

    // Mengirim response sukses beserta data post
    yang baru dibuat.
    return $this->_response(true, 'Post created
    successfully', $this->Post_model->get_by_id($post_id),
    201);
}

// Endpoint untuk mengubah data post berdasarkan
ID.
public function update($id)
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengecek token JWT sebelum mengubah data.
    if (!$this->_auth_payload()) return;

    // Mengecek apakah post dengan ID tersebut ada.
    if (!$this->Post_model->exists($id)) {
        // Mengirim response error jika post tidak
        ditemukan.
        return $this->_response(false, 'Post not
        found', null, 404);
    }

    // Membaca request body JSON lalu mengubahnya
    menjadi array.
    $input =
    json_decode(file_get_contents('php://input'), true);
    // Menyiapkan array kosong untuk data yang akan
    diupdate.
    $data = array();
    // Mengulang field yang boleh diupdate.
    foreach (array('title', 'author', 'article') as
    $field) {
        // Mengecek apakah field tersebut dikirim
        pada input.

```



```

        if (isset($input[$field])) {
            // Memasukkan nilai field ke array
update.
            $data[$field] = $input[$field];
        }
    }

    // Mengecek apakah tidak ada data yang dikirim
    untuk diupdate.
    if (empty($data)) {
        // Mengirim response error jika request
    tidak membawa data update.
        return $this->_response(false, 'No data to
update', null, 400);
    }

    // Menjalankan update data melalui model.
    $this->Post_model->update($id, $data);
    // Mengirim response sukses beserta data
    terbaru.
    return $this->_response(true, 'Post updated
successfully', $this->Post_model->get_by_id($id));
}

// Endpoint untuk menghapus data post berdasarkan
ID.
public function delete($id)
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengecek token JWT sebelum menghapus data.
    if (!$this->_auth_payload()) return;

    // Mengecek apakah post dengan ID tersebut ada.
    if (!$this->Post_model->exists($id)) {
        // Mengirim response error jika post tidak
    ditemukan.

```

```

        return $this->_response(false, 'Post not
found', null, 404);
    }

    // Menghapus data post dari database melalui
model.
    $this->Post_model->delete($id);
    // Mengirim response sukses setelah data
dihapus.
    return $this->_response(true, 'Post deleted
successfully');
}

    // Method private untuk mengambil dan memverifikasi
token JWT.
    private function _auth_payload()
    {
        // Mengambil token dari header Authorization
request.
        $token = $this->jwt->get_token_from_request();
        // Memverifikasi token dan mengambil payload-
nya.
        $payload = $this->jwt->verify($token);
        // Mengecek apakah token tidak valid.
        if (!$payload) {
            // Mengirim response unauthorized jika
token salah atau kosong.
            $this->_response(false, 'Unauthorized or
invalid token', null, 401);
            // Mengembalikan false sebagai tanda
autentikasi gagal.
            return false;
        }
        // Mengembalikan payload jika token valid.
        return $payload;
    }
}

```

```

    // Method private untuk mengatur response menjadi
JSON.
    private function _json()
    {
        // Membersihkan output buffer agar JSON tidak
tercampur output lain.
        ob_clean();
        // Mengatur header content type sebagai JSON
UTF-8.
        header('Content-Type: application/json;
charset=utf-8');
    }

    // Method private untuk mengirim response JSON
standar.
    private function _response($status, $message, $data
= null, $code = 200)
    {
        // Mengatur HTTP status code response.
        http_response_code($code);
        // Membuat struktur dasar response API.
        $response = array('status' => $status,
'message' => $message);
        // Mengecek apakah ada data tambahan yang perlu
dikirim.
        if ($data !== null) {
            // Menambahkan data ke response.
            $response['data'] = $data;
        }
        // Mencetak response dalam format JSON.
        echo json_encode($response,
JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        // Menghentikan eksekusi agar tidak ada output
tambahan.
        exit;
    }
}

```

Post_model

```
<?php
// Mencegah file model diakses langsung dari browser.
defined('BASEPATH') OR exit('No direct script access
allowed');

// Model untuk mengelola data post pada API.
class Post_model extends CI_Model {

    // Nama tabel database yang digunakan untuk
    menyimpan data post.
    private $table = 'posts';

    // Constructor dijalankan otomatis saat model
    dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan CI_Model.
        parent::__construct();
        // Memuat koneksi database CodeIgniter.
        $this->load->database();
    }

    // Mengambil semua data post dari database.
    public function get_all()
    {
        // Mengurutkan data dari ID terbaru lalu
        mengembalikan hasil sebagai array object.
        return $this->db->order_by('id', 'DESC')->
>get($this->table)->result();
    }

    // Mengambil satu data post berdasarkan ID.
    public function get_by_id($id)
    {
        // Menjalankan query SELECT * FROM posts WHERE
        id = $id dan mengambil satu baris.
```

```

        return $this->db->get_where($this->table,
array('id' => $id))->row();
    }

    // Menambahkan post baru ke database.
    public function create($data)
    {
        // Mengisi waktu data dibuat.
        $data['created_at'] = date('Y-m-d H:i:s');
        // Mengisi waktu data terakhir diubah.
        $data['updated_at'] = date('Y-m-d H:i:s');
        // Menjalankan query INSERT ke tabel posts.
        $this->db->insert($this->table, $data);
        // Mengembalikan ID post yang baru dibuat.
        return $this->db->insert_id();
    }

    // Mengubah data post berdasarkan ID.
    public function update($id, $data)
    {
        // Memperbarui waktu update setiap kali data
        diubah.
        $data['updated_at'] = date('Y-m-d H:i:s');
        // Menentukan data mana yang akan diupdate
        berdasarkan ID.
        $this->db->where('id', $id);
        // Menjalankan query UPDATE dan mengembalikan
        status berhasil/gagal.
        return $this->db->update($this->table, $data);
    }

    // Menghapus data post berdasarkan ID.
    public function delete($id)
    {
        // Menentukan data mana yang akan dihapus
        berdasarkan ID.
        $this->db->where('id', $id);

```

```

        // Menjalankan query DELETE dan mengembalikan
status berhasil/gagal.
        return $this->db->delete($this->table);
    }

    // Mengecek apakah data post dengan ID tertentu
ada.
    public function exists($id)
    {
        // Mengembalikan TRUE jika jumlah data dengan
ID tersebut lebih dari 0.
        return $this->db->get_where($this->table,
array('id' => $id))->num_rows() > 0;
    }
}

```

User_model

```

<?php
// Mencegah file model diakses langsung dari browser.
defined('BASEPATH') OR exit('No direct script access
allowed');

// Model untuk mengelola data user pada API
authentication.
class User_model extends CI_Model {

    // Nama tabel database yang digunakan untuk
menyimpan data user.
    private $table = 'users';

    // Constructor dijalankan otomatis saat model
dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan CI_Model.
        parent::__construct();
        // Memuat koneksi database CodeIgniter.
    }
}

```

```

        $this->load->database();
    }

    // Menambahkan user baru ke database.
    public function create($data)
    {
        // Mengisi waktu data dibuat.
        $data['created_at'] = date('Y-m-d H:i:s');
        // Mengisi waktu data terakhir diubah.
        $data['updated_at'] = date('Y-m-d H:i:s');
        // Menjalankan query INSERT ke tabel users.
        $this->db->insert($this->table, $data);
        // Mengembalikan ID user yang baru dibuat.
        return $this->db->insert_id();
    }

    // Mengambil data user berdasarkan ID.
    public function get_by_id($id)
    {
        // Menjalankan query SELECT * FROM users WHERE
        id = $id dan mengambil satu baris.
        return $this->db->get_where($this->table,
array('id' => $id))->row();
    }

    // Mengambil data user berdasarkan email.
    public function get_by_email($email)
    {
        // Mengubah email menjadi huruf kecil agar
        pencarian konsisten.
        return $this->db->get_where($this->table,
array('email' => strtolower($email)))->row();
    }

    // Mengecek apakah email sudah terdaftar di
    database.
    public function email_exists($email)
    {

```

```

        // Mengembalikan TRUE jika jumlah data dengan
        email tersebut lebih dari 0.
        return $this->db->get_where($this->table,
array('email' => strtolower($email)))->num_rows() > 0;
    }
}

```

Routes

```

<?php
defined('BASEPATH') OR exit('No direct script access
allowed');

/*
| -----
|
| URI ROUTING
| -----
|
| This file lets you re-map URI requests to specific
controller functions.
|
| Typically there is a one-to-one relationship between
a URL string
| and its corresponding controller class/method. The
segments in a
| URL normally follow this pattern:
|
|    example.com/class/method/id/
|
| In some instances, however, you may want to remap
this relationship
| so that a different class/function is called than the
one
| corresponding to the URL.
|
| Please see the user guide for complete details:
|

```



```
| http://codeigniter.com/user\_guide/general/routing.h  
tml  
|  
| -----  
-----  
| RESERVED ROUTES  
| -----  
-----  
|  
| There are three reserved routes:  
|  
| $route['default_controller'] = 'welcome';  
|  
| This route indicates which controller class should be  
loaded if the  
| URI contains no data. In the above example, the  
"welcome" class  
| would be loaded.  
|  
| $route['404_override'] = 'errors/page_missing';  
|  
| This route will tell the Router which  
controller/method to use if those  
| provided in the URL cannot be matched to a valid  
route.  
|  
| $route['translate_uri_dashes'] = FALSE;  
|  
| This is not exactly a route, but allows you to  
automatically route  
| controller and method names that contain dashes. '-'  
isn't a valid  
| class or method name character, so it requires  
translation.  
| When you set this option to TRUE, it will replace ALL  
dashes in the  
| controller and method URI segments.  
|
```

```

| Examples: my-controller/index -> my_controller/index
|           my-controller/my-method ->
my_controller/my_method
*/
$route['default_controller'] = 'Crudjs/Crudjs';
$route['404_override'] = '';
$route['translate_uri_dashes'] = FALSE;

$route['login'] = 'auth';
$route['register'] = 'auth/register';
$route['logout'] = 'auth/logout';
$route['dashboard'] = 'dashboard';
$route['peserta'] = 'peserta';
$route['register_API'] = 'Register_API/register_API';
$route['peserta/simpan'] = 'peserta/save';
$route['peserta/hapus/(:num)'] = 'peserta/delete/$1';

// HMVC AJAX CRUD routes
$route['crudjs'] = 'Crudjs/Crudjs';
$route['crudjs/debug'] = 'Crudjs/Crudjs/debug';
$route['crudjs/test'] = 'Crudjs/Crudjs/test';
$route['crudjs/get_all'] = 'Crudjs/Crudjs/get_all';
$route['crudjs/get_record/(:num)'] =
'Crudjs/Crudjs/get_record/$1';
$route['crudjs/store'] = 'Crudjs/Crudjs/store';
$route['crudjs/update/(:num)'] =
'Crudjs/Crudjs/update/$1';
$route['crudjs/delete/(:num)'] =
'Crudjs/Crudjs/delete/$1';

// API Routes
$route['api/auth/register'] = 'Api/Auth/register'; //
Endpoint API untuk mendaftarkan user baru melalui
controller Api/Auth method register.
$route['api/auth/login'] = 'Api/Auth/login'; //
Endpoint API untuk proses login user melalui controller
Api/Auth method login.

```

```

$route['api/auth/logout'] = 'Api/Auth/logout'; //
Endpoint API untuk proses logout user melalui
controller Api/Auth method logout.
$route['api/auth/me'] = 'Api/Auth/me'; // Endpoint API
untuk mengambil data user yang sedang login melalui
controller Api/Auth method me.
$route['api/post/(:num)'] = 'Api/Post/handle/$1'; //
Endpoint API untuk mengelola satu data post berdasarkan
ID angka yang dikirim pada URL.
$route['api/post'] = 'Api/Post/handle'; // Endpoint API
untuk mengelola data post tanpa ID, misalnya
menampilkan semua data atau menambah post baru.

```

Crudjs

```

<?php
// Mencegah file controller diakses langsung tanpa
lewat framework CodeIgniter.
defined('BASEPATH') OR exit('No direct script access
allowed');

// Controller HMVC untuk fitur CRUD with AJAX.
class Crudjs extends MX_Controller {

    // Constructor dijalankan otomatis saat controller
dipanggil.
    public function __construct()
    {
        // Memanggil constructor dari MX_Controller.
        parent::__construct();
        // Memuat model Crudjs_model untuk akses
database.
        $this->load->model('Crudjs_model');
        // Memuat helper URL dan form.
        $this->load->helper(array('url', 'form'));
        // Memuat library validasi form, upload file,
dan session.

```

```

        $this->load->library(array('form_validation',
'upload', 'session'));
    }

    // Method untuk mengecek kondisi database, tabel,
folder upload, dan sample data.
    public function debug()
    {
        // Menentukan folder upload gambar post.
        $upload_path = './uploads/posts/';
        // Menyusun informasi debug dalam bentuk array.
        $debug_info = array(
            // Informasi koneksi database.
            'database' => array(
                // Mengecek apakah database berhasil
terkoneksi.
                'connected' => $this->db->conn_id ?
'Yes' : 'No',
                // Menampilkan nama database yang
sedang digunakan.
                'database' => $this->db->database
            ),
            // Mengecek apakah tabel posts ada di
database.
            'table_exists' => $this->db-
>table_exists('posts') ? 'Yes' : 'No',
            // Informasi folder upload.
            'upload_folder' => array(
                // Mengecek apakah folder upload
tersedia.
                'exists' => is_dir($upload_path) ?
'Yes' : 'No',
                // Mengecek apakah folder upload bisa
ditulis.
                'writable' => is_writable($upload_path)
? 'Yes' : 'No'
            ),

```

```

        // Tempat menyimpan contoh data dari
database.
        'sample_records' => array()
    );

    // Mencoba mengambil data untuk keperluan
debug.
    try {
        // Mengambil semua data post dari model.
        $records = $this->Crudjs_model->get_all();
        // Mengecek apakah data tidak kosong.
        if (!empty($records)) {
            // Menyimpan jumlah data dan data
pertama sebagai contoh.
            $debug_info['sample_records'] = array(
                // Menghitung jumlah record.
                'count' => count($records),
                // Mengambil record pertama untuk
contoh.
                'first_record' => $records[0]
            );
        }
        // Menangkap error jika proses pengambilan data
gagal.
    } catch (Exception $e) {
        // Menyimpan pesan error ke array debug.
        $debug_info['sample_records']['error'] =
$e->getMessage();
    }

    // Menampilkan informasi debug dalam format
JSON yang rapi.
    echo '<pre>' . json_encode($debug_info,
JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES) . '</pre>';
}

    // Method untuk menampilkan halaman utama CRUD with
AJAX.

```

```

public function index()
{
    // Mengirim judul halaman ke view.
    $data['title'] = 'CRUD with AJAX';
    // Memuat view utama CRUD AJAX.
    $this->load->view('crudjs/index', $data);
}

// Method AJAX untuk mengambil semua data post.
public function get_all()
{
    // Mengatur response menjadi JSON.
    $this->_json();

    // Mencoba mengambil data dari database.
    try {
        // Mengambil semua data melalui model.
        $records = $this->Crudjs_model->get_all();
        // Mengirim response sukses beserta data
dalam format JSON.
        echo json_encode(array('status' =>
'success', 'data' => $records), JSON_UNESCAPED_SLASHES
| JSON_UNESCAPED_UNICODE);
        // Menangkap error jika data gagal diambil.
    } catch (Exception $e) {
        // Mengirim response error dalam format
JSON.
        echo json_encode(array('status' => 'error',
'message' => 'Failed to load records: ' . $e-
>getMessage()));
    }
    // Menghentikan eksekusi agar tidak ada output
tambahan.
    exit;
}

// Method AJAX untuk mengambil satu data post
berdasarkan ID.

```

```

public function get_record($id)
{
    // Mengatur response menjadi JSON.
    $this->_json();

    // Mencoba mengambil satu data dari database.
    try {
        // Mengecek apakah data dengan ID tersebut
ada.
        if (!$this->Crudjs_model->exists($id)) {
            // Mengirim response error jika data
tidak ditemukan.
            echo json_encode(array('status' =>
'error', 'message' => 'Record not found'));
            // Menghentikan eksekusi setelah
response dikirim.
            exit;
        }

        // Mengirim response sukses berisi detail
data post.
        echo json_encode(array(
            // Status response berhasil.
            'status' => 'success',
            // Data post berdasarkan ID.
            'data' => $this->Crudjs_model-
>get_by_id($id)
        ), JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE);
        // Menangkap error jika proses ambil data
gagal.
    } catch (Exception $e) {
        // Mengirim response error dalam format
JSON.
        echo json_encode(array('status' => 'error',
'message' => 'Failed to load record: ' . $e-
>getMessage()));
    }
}

```

```

        // Menghentikan eksekusi agar output tetap
bersih.
        exit;
    }

    // Method AJAX untuk menyimpan data post baru.
    public function store()
    {
        // Mengatur response menjadi JSON.
        $this->_json();

        // Mencoba menjalankan proses tambah data.
        try {
            // Validasi title wajib diisi dan maksimal
255 karakter.
            $this->form_validation->set_rules('title',
'Title', 'required|max_length[255]');
            // Validasi author wajib diisi dan maksimal
255 karakter.
            $this->form_validation->set_rules('author',
'Author', 'required|max_length[255]');
            // Validasi article wajib diisi.
            $this->form_validation-
>set_rules('article', 'Article', 'required');

            // Mengecek apakah validasi form gagal.
            if ($this->form_validation->run() ===
FALSE) {
                // Mengirim pesan error validasi dalam
format JSON.
                echo json_encode(array('status' =>
'error', 'message' => 'Validation error: ' .
strip_tags(validation_errors())));
                // Menghentikan proses simpan data.
                exit;
            }

```



```

        // Menyusun data yang akan dimasukkan ke
database.
        $data = array(
            // Mengambil input title dari form
POST.
            'title' => $this->input->post('title'),
            // Mengambil input author dari form
POST.
            'author' => $this->input-
>post('author'),
            // Mengambil input article dari form
POST.
            'article' => $this->input-
>post('article'),
            // Mengisi waktu pembuatan data.
            'created_at' => date('Y-m-d H:i:s'),
            // Mengisi waktu update awal data.
            'updated_at' => date('Y-m-d H:i:s')
        );

        // Mengecek apakah user mengupload gambar.
        if (!empty($_FILES['image']['name'])) {
            // Memanggil method upload gambar.
            $upload_result = $this-
>_upload_image();
            // Mengecek apakah upload berhasil.
            if ($upload_result['status']) {
                // Menyimpan path gambar ke data
yang akan dimasukkan.
                $data['image'] = 'posts/' .
$upload_result['file_name'];
                // Jika upload gagal.
            } else {
                // Mengirim response error upload.
                echo json_encode(array('status' =>
'error', 'message' => $upload_result['error']));
                // Menghentikan proses simpan data.
                exit;
            }
        }
    }
}

```

```

    }
}

// Menyimpan data ke database dan mengambil
ID data baru.
$record_id = $this->Crudjs_model-
>insert($data);
// Mengecek apakah proses insert berhasil.
if ($record_id) {
    // Mengirim response sukses beserta
data yang baru dibuat.
    echo json_encode(array(
        // Status response berhasil.
        'status' => 'success',
        // Pesan sukses untuk ditampilkan
di halaman.
        'message' => 'Record created
successfully!',
        // Mengambil kembali data baru
berdasarkan ID.
        'data' => $this->Crudjs_model-
>get_by_id($record_id)
    ), JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE);
    // Jika insert gagal.
} else {
    // Mengirim response error gagal
membuat data.
    echo json_encode(array('status' =>
'error', 'message' => 'Failed to create record'));
}
// Menangkap error jika ada exception saat
tambah data.
} catch (Exception $e) {
    // Mengirim response error exception.
    echo json_encode(array('status' => 'error',
'message' => 'Exception error: ' . $e->getMessage()));
}

```

```

        // Menghentikan eksekusi setelah response JSON
dikirim.
        exit;
    }

    // Method AJAX untuk mengubah data post berdasarkan
ID.
    public function update($id)
    {
        // Mengatur response menjadi JSON.
        $this->_json();

        // Mencoba menjalankan proses update data.
        try {
            // Mengecek apakah data dengan ID tersebut
ada.
            if (!$this->Crudjs_model->exists($id)) {
                // Mengirim response error jika data
tidak ditemukan.
                echo json_encode(array('status' =>
'error', 'message' => 'Record not found'));
                // Menghentikan proses update.
                exit;
            }

            // Validasi title jika dikirim, maksimal
255 karakter.
            $this->form_validation->set_rules('title',
'Title', 'max_length[255]');
            // Validasi author jika dikirim, maksimal
255 karakter.
            $this->form_validation->set_rules('author',
'Author', 'max_length[255]');

            // Mengecek apakah validasi gagal.
            if ($this->form_validation->run() ===
FALSE) {
                // Mengirim response error validasi.

```

```

        echo json_encode(array('status' =>
'error', 'message' => 'Validation error: ' .
strip_tags(validation_errors())));
        // Menghentikan proses update.
        exit;
    }

    // Membuat data awal update berisi waktu
update terbaru.
    $data = array('updated_at' => date('Y-m-d
H:i:s'));
    // Mengulang field yang boleh diupdate.
    foreach (array('title', 'author',
'article') as $field) {
        // Mengecek apakah field dikirim dari
form.
        if ($this->input->post($field)) {
            // Memasukkan nilai field ke array
data update.
            $data[$field] = $this->input-
>post($field);
        }
    }

    // Mengecek apakah ada gambar baru yang
diupload.
    if (!empty($_FILES['image']['name'])) {
        // Mengambil data lama untuk mengecek
gambar sebelumnya.
        $old_record = $this->Crudjs_model-
>get_by_id($id);
        // Mengecek apakah data lama memiliki
gambar.
        if ($old_record && $old_record->image)
        {
            // Menyusun path gambar lama.
            $old_image_path = './uploads/' .
$old_record->image;

```

```

        // Mengecek apakah file gambar lama
ada.
        if (file_exists($old_image_path)) {
            // Menghapus gambar lama agar
tidak menumpuk di folder upload.
            unlink($old_image_path);
        }
    }

    // Mengupload gambar baru.
    $upload_result = $this->_upload_image();
    // Mengecek apakah upload gambar baru
berhasil.
    if ($upload_result['status']) {
        // Menyimpan path gambar baru ke
data update.
        $data['image'] = 'posts/' .
$upload_result['file_name'];
        // Jika upload gambar baru gagal.
    } else {
        // Mengirim response error upload.
        echo json_encode(array('status' =>
'error', 'message' => $upload_result['error']));
        // Menghentikan proses update.
        exit;
    }
}

// Menjalankan update data melalui model.
if ($this->Crudjs_model->update($id,
$data)) {
    // Mengirim response sukses beserta
data terbaru.
    echo json_encode(array(
        // Status response berhasil.
        'status' => 'success',
        // Pesan sukses update.

```

```

        'message' => 'Record updated
successfully!',
        // Mengambil data terbaru
berdasarkan ID.
        'data' => $this->Crudjs_model-
>get_by_id($id)
    ), JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE);
    // Jika update gagal.
    } else {
        // Mengirim response error gagal
update.
        echo json_encode(array('status' =>
'error', 'message' => 'Failed to update record'));
    }
    // Menangkap error jika ada exception saat
update.
    } catch (Exception $e) {
        // Mengirim response error exception.
        echo json_encode(array('status' => 'error',
'message' => 'Exception error: ' . $e->getMessage()));
    }
    // Menghentikan eksekusi setelah response JSON
dikirim.
    exit;
}

// Method AJAX untuk menghapus data post
berdasarkan ID.
public function delete($id)
{
    // Mengatur response menjadi JSON.
    $this->_json();

    // Mencoba menjalankan proses hapus data.
    try {
        // Mengambil data post berdasarkan ID.

```

```

        $record = $this->Crudjs_model-
>get_by_id($id);
        // Mengecek apakah data ditemukan.
        if (!$record) {
            // Mengirim response error jika data
tidak ditemukan.
            echo json_encode(array('status' =>
'error', 'message' => 'Record not found'));
            // Menghentikan proses hapus.
            exit;
        }

        // Mengecek apakah data memiliki gambar.
        if ($record->image) {
            // Menyusun path file gambar yang akan
dihapus.
            $image_path = './uploads/' . $record-
>image;
            // Mengecek apakah file gambar benar-
benar ada.
            if (file_exists($image_path)) {
                // Menghapus file gambar dari
folder upload.
                unlink($image_path);
            }
        }

        // Menghapus data dari database melalui
model.
        if ($this->Crudjs_model->delete($id)) {
            // Mengirim response sukses jika data
berhasil dihapus.
            echo json_encode(array('status' =>
'success', 'message' => 'Record deleted
successfully!'));
            // Jika proses delete database gagal.
        } else {

```

```

        // Mengirim response error gagal hapus
data.
        echo json_encode(array('status' =>
'error', 'message' => 'Failed to delete record'));
    }
    // Menangkap error jika ada exception saat
hapus data.
    } catch (Exception $e) {
        // Mengirim response error exception.
        echo json_encode(array('status' => 'error',
'message' => 'Exception error: ' . $e->getMessage()));
    }
    // Menghentikan eksekusi setelah response JSON
dikirim.
    exit;
}

// Method untuk mengetes apakah endpoint AJAX/API
CRUD berjalan.
public function test()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengirim response sukses sederhana.
    echo json_encode(array('status' => 'success',
'message' => 'API is working'));
    // Menghentikan eksekusi setelah response
dikirim.
    exit;
}

// Method private untuk mengupload gambar post.
private function _upload_image()
{
    // Menentukan folder tujuan upload.
    $upload_path = './uploads/posts/';
    // Mengecek apakah folder upload belum ada.
    if (!is_dir($upload_path)) {

```



```

        // Membuat folder upload beserta
        subfoldernya jika belum ada.
        mkdir($upload_path, 0777, true);
    }

    // Mengambil nama asli file gambar dari input
    upload.
    $original_name = $_FILES['image']['name'];
    // Membersihkan nama file dari karakter yang
    tidak aman.
    $sanitized_name = preg_replace('/[^A-Za-z0-9._-
]/', '_', $original_name);
    // Membuat nama file unik dengan tambahan
    timestamp.
    $file_name = time() . '_' . $sanitized_name;

    // Menentukan folder tujuan upload.
    $config['upload_path'] = $upload_path;
    // Menentukan tipe file gambar yang diizinkan.
    $config['allowed_types'] = 'jpg|jpeg|png';
    // Menentukan ukuran maksimal file dalam KB.
    $config['max_size'] = 2048;
    // Menentukan nama file hasil upload.
    $config['file_name'] = $file_name;
    // Mencegah file lama ditimpa jika ada nama
    yang sama.
    $config['overwrite'] = FALSE;

    // Menginisialisasi library upload dengan
    konfigurasi di atas.
    $this->upload->initialize($config);

    // Menjalankan proses upload dari input bernama
    image.
    if ($this->upload->do_upload('image')) {
        // Mengambil informasi file yang berhasil
        diupload.
        $upload_data = $this->upload->data();
    }

```

```

        // Mengembalikan status berhasil dan nama
file.
        return array('status' => TRUE, 'file_name'
=> $upload_data['file_name']);
    }

    // Mengembalikan status gagal dan pesan error
upload.
    return array('status' => FALSE, 'error' =>
$this->upload->display_errors('', ''));
}

    // Method private untuk mengatur header response
JSON.
    private function _json()
    {
        // Membersihkan output buffer agar JSON tidak
tercampur output lain.
        ob_clean();
        // Mengatur content type response menjadi JSON
UTF-8.
        header('Content-Type: application/json;
charset=utf-8');
    }
}

```

Crudjs_model

```

<?php
// Mencegah file diakses langsung dari browser tanpa
lewat CodeIgniter.
defined('BASEPATH') OR exit('No direct script access
allowed');

// Model untuk mengatur query database pada fitur CRUD
with AJAX.
class Crudjs_model extends CI_Model {

```

```

    // Nama tabel database yang digunakan untuk
    menyimpan data post.
    private $table = 'posts';

    // Constructor dijalankan otomatis saat model
    dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan CI_Model.
        parent::__construct();
        // Memuat koneksi database CodeIgniter.
        $this->load->database();
    }

    // Mengambil semua data post dari tabel posts.
    public function get_all()
    {
        // Menjalankan query SELECT * FROM posts.
        $query = $this->db->get($this->table);
        // Mengubah hasil query menjadi array object.
        $records = $query->result();

        // Melakukan perulangan untuk setiap data post.
        foreach ($records as $record) {
            // Mengecek apakah data post memiliki
gambar.
            if ($record->image) {
                // Menghapus prefix folder posts/ agar
path gambar tidak dobel.
                $imagePath = str_replace('posts/', '',
$record->image);
                // Membuat URL lengkap gambar agar bisa
ditampilkan di halaman AJAX.
                $record->image_url =
base_url('uploads/posts/' . $imagePath);
            } else {
                // Jika tidak ada gambar, nilai
image_url dibuat null.

```

```

        $record->image_url = null;
    }
}

// Mengembalikan semua data post ke controller.
return $records;
}

// Mengambil satu data post berdasarkan ID.
public function get_by_id($id)
{
    // Menjalankan query SELECT * FROM posts WHERE
    id = $id.
    $query = $this->db->get_where($this->table,
    array('id' => $id));
    // Mengambil satu baris hasil query sebagai
    object.
    $record = $query->row();

    // Mengecek apakah data ditemukan dan memiliki
    gambar.
    if ($record && $record->image) {
        // Menghapus prefix folder posts/ agar path
        gambar rapi.
        $imagePath = str_replace('posts/', '',
        $record->image);
        // Membuat URL lengkap gambar untuk dikirim
        ke JavaScript.
        $record->image_url =
        base_url('uploads/posts/' . $imagePath);
    }

    // Mengembalikan satu data post ke controller.
    return $record;
}

// Menambahkan data post baru ke database.
public function insert($data)

```

```

    {
        // Menjalankan query INSERT ke tabel posts.
        $this->db->insert($this->table, $data);
        // Mengembalikan ID dari data yang baru
        ditambahkan.
        return $this->db->insert_id();
    }

    // Mengubah data post berdasarkan ID.
    public function update($id, $data)
    {
        // Menentukan data mana yang akan diupdate
        berdasarkan kolom id.
        $this->db->where('id', $id);
        // Menjalankan query UPDATE dan mengembalikan
        status berhasil/gagal.
        return $this->db->update($this->table, $data);
    }

    // Menghapus data post berdasarkan ID.
    public function delete($id)
    {
        // Menentukan data mana yang akan dihapus
        berdasarkan kolom id.
        $this->db->where('id', $id);
        // Menjalankan query DELETE dan mengembalikan
        status berhasil/gagal.
        return $this->db->delete($this->table);
    }

    // Mengecek apakah data dengan ID tertentu ada di
    database.
    public function exists($id)
    {
        // Mengambil data dari tabel posts berdasarkan
        ID.
        $query = $this->db->get_where($this->table,
        array('id' => $id));
    }

```

```

        // Mengembalikan TRUE jika jumlah baris lebih
dari 0.
        return $query->num_rows() > 0;
    }
}

```

Views

Footer

<script>

// Menyimpan URL dasar endpoint CRUDJS agar fetch AJAX lebih ringkas.

const baseUrl = "<?php echo base_url('crudjs'); ?>";

// Menjalankan kode setelah seluruh struktur HTML selesai dimuat.

document.addEventListener('DOMContentLoaded', function () {

// Mengambil data post pertama kali saat halaman dibuka.

loadRecords();

// Menambahkan event submit pada form agar disimpan lewat AJAX.

document.getElementById('postForm').addEventListener('submit',

savePost;

});

// Fungsi untuk menampilkan pesan notifikasi sukses atau error.

function showAlert(message, type) {

// Mengambil elemen alert dari halaman.

const alert = document.getElementById('alert');

// Mengatur class alert sesuai tipe pesan.

alert.className = 'alert alert-' + type;

// Mengisi teks pesan ke dalam alert.

alert.innerHTML = message;

// Menampilkan alert.

alert.style.display = 'block';

// Menyembunyikan alert otomatis setelah 3 detik.

setTimeout(() => alert.style.display = 'none', 3000);

}

// Fungsi AJAX untuk mengambil semua data post dari server.

function loadRecords() {

// Menampilkan indikator loading.

document.getElementById('loading').style.display = 'block';

// Mengirim request ke endpoint crudjs/get_all.

```

        fetch(baseUrl + '/get_all')
    // Mengubah response server menjadi JSON.
    .then(response => response.json())
    // Memproses data JSON yang diterima.
    .then(result => {
// Menyembunyikan indikator loading setelah response diterima.
document.getElementById('loading').style.display = 'none';
    // Mengambil elemen tbody tabel.
const tbody = document.getElementById('postsBody');
    // Mengosongkan isi tabel sebelum diisi data terbaru.
tbody.innerHTML = "";

// Mengecek apakah response sukses dan memiliki data.
if (result.status === 'success' && result.data.length > 0) {
    // Melakukan perulangan untuk setiap data post.
    result.data.forEach(row => {
    // Menambahkan baris tabel baru untuk setiap post.
tbody.innerHTML += `
        <tr>
            <td>${row.id}</td>
            <td>${row.image_url ? `` : '-'}</td>
            <td>${escapeHtml(row.title || "")}</td>
            <td>${escapeHtml(row.author || "")}</td>
            <td>${escapeHtml((row.article || "").substring(0, 100))}</td>
            <td>${row.created_at || '-'}</td>
            <td class="actions">
                <button class="btn btn-warning"
                    onclick="editPost(${row.id})">Edit</button>
                <button class="btn btn-danger"
                    onclick="deletePost(${row.id})">Delete</button>
            </td>
        </tr>
    `;
    });

    // Jika data kosong atau response bukan success.
    } else {
// Menampilkan baris keterangan bahwa belum ada data.
tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;">Belum ada data</td></tr>';
    }
}

```

```

    })
    // Menangkap error jika request AJAX gagal.
    .catch(error => {
        // Menyembunyikan indikator loading.
        document.getElementById('loading').style.display = 'none';
        // Menampilkan pesan error ke user.
        showAlert('Error loading records: ' + error.message, 'error');
    });
}

// Fungsi untuk membuka modal tambah data.
function openModal() {
    // Mengatur judul modal menjadi Tambah Post.
    document.getElementById('modalTitle').innerText = 'Tambah Post';
    // Mengosongkan semua input form.
    document.getElementById('postForm').reset();
    // Mengosongkan ID agar form dianggap sebagai tambah data.
    document.getElementById('postId').value = '';
    // Mengosongkan preview gambar lama.
    document.getElementById('currentImage').innerHTML = '';
    // Menampilkan modal ke layar.
    document.getElementById('postModal').style.display = 'block';
}

// Fungsi untuk menutup modal.
function closeModal() {
    // Menyembunyikan modal dari layar.
    document.getElementById('postModal').style.display = 'none';
}

// Fungsi untuk menyimpan data tambah atau edit menggunakan AJAX.
function savePost(event) {
    // Mencegah form reload halaman secara default.
    event.preventDefault();
    // Mengambil nilai ID untuk menentukan tambah atau update.
    const id = document.getElementById('postId').value;
    // Mengambil semua data form termasuk file gambar.
    const formData = new FormData(document.getElementById('postForm'));
    // Menentukan URL tujuan: update jika ada ID, store jika tidak ada ID.
    const url = id ? baseUrl + '/update/' + id : baseUrl + '/store';

```



```

// Mengirim data form ke server menggunakan method POST.
fetch(url, { method: 'POST', body: formData })
// Mengubah response server menjadi JSON.
.then(response => response.json())
// Memproses hasil simpan/update dari server.
.then(result => {
// Mengecek apakah proses simpan/update berhasil.
if (result.status === 'success') {
// Menutup modal setelah sukses.
closeModal();
// Memuat ulang data tabel agar data terbaru tampil.
loadRecords();
// Menampilkan pesan sukses.
showAlert(result.message, 'success');
// Jika server mengirim status error.
} else {
// Menampilkan pesan error dari server.
showAlert(result.message, 'error');
}
})
// Menangkap error jika request AJAX gagal.
.catch(error => showAlert('Error saving record: ' + error.message, 'error'));
}

```

```

// Fungsi untuk mengambil data yang akan diedit berdasarkan ID.
function editPost(id) {
// Mengirim request ke endpoint get_record dengan ID post.
fetch(baseUrl + '/get_record/' + id)
// Mengubah response server menjadi JSON.
.then(response => response.json())
// Memproses data post yang diterima.
.then(result => {
// Mengecek apakah data berhasil ditemukan.
if (result.status === 'success') {
// Menyimpan data post ke variabel row.
const row = result.data;
// Mengubah judul modal menjadi Edit Post.
document.getElementById('modalTitle').innerText = 'Edit Post';
// Mengisi input hidden ID dengan ID post.
document.getElementById('postId').value = row.id;
// Mengisi input title dengan data dari server.

```

```

        document.getElementById('title').value = row.title || "";
        // Mengisi input author dengan data dari server.
        document.getElementById('author').value = row.author || "";
        // Mengisi textarea article dengan data dari server.
        document.getElementById('article').value = row.article || "";
        // Menampilkan preview gambar jika post memiliki gambar.
        document.getElementById('currentImage').innerHTML =
row.image_url ? `` : "";
        // Menampilkan modal edit.
        document.getElementById('postModal').style.display = 'block';
        // Jika data gagal ditemukan.
        } else {
            // Menampilkan pesan error dari server.
            showAlert(result.message, 'error');
        }
    });
}

```

```

// Fungsi untuk menghapus post berdasarkan ID.
function deletePost(id) {
    // Menampilkan konfirmasi sebelum data dihapus.
    if (!confirm('Yakin ingin menghapus data ini?')) return;

    // Mengirim request hapus ke endpoint delete dengan method POST.
    fetch(baseUrl + '/delete/' + id, { method: 'POST' })
        // Mengubah response server menjadi JSON.
        .then(response => response.json())
        // Memproses hasil hapus dari server.
        .then(result => {
            // Mengecek apakah data berhasil dihapus.
            if (result.status === 'success') {
                // Memuat ulang data tabel setelah hapus.
                loadRecords();
                // Menampilkan pesan sukses.
                showAlert(result.message, 'success');
                // Jika hapus gagal.
            } else {
                // Menampilkan pesan error dari server.
                showAlert(result.message, 'error');
            }
        })
    }
}

```

```

        // Menangkap error jika request AJAX gagal.
        .catch(error => showAlert('Error deleting record: ' + error.message,
                                'error'));
    }

```

```

// Fungsi untuk mengamankan teks agar tidak menjalankan HTML/script
berbahaya.

```

```

    function escapeHtml(text) {
        // Membuat elemen div sementara.
        const div = document.createElement('div');
        // Memasukkan teks sebagai textContent agar karakter HTML di-escape.
        div.textContent = text;
        // Mengembalikan teks yang sudah aman dalam bentuk HTML.
        return div.innerHTML;
    }

```

```

// Event global untuk menutup modal saat area luar modal diklik.

```

```

    window.onclick = function (event) {
        // Mengambil elemen modal.
        const modal = document.getElementById('postModal');
        // Mengecek apakah area yang diklik adalah background modal.
        if (event.target === modal) {
            // Menutup modal jika user klik area luar konten modal.
            closeModal();
        }
    };
</script>
</body>

```

```

</html><script>
// Menyimpan URL dasar endpoint CRUDJS agar fetch AJAX
lebih ringkas.
const baseUrl = "<?php echo base_url('crudjs'); ?>";

// Menjalankan kode setelah seluruh struktur HTML
selesai dimuat.
document.addEventListener('DOMContentLoaded', function
() {
    // Mengambil data post pertama kali saat halaman
dibuka.

```

```

    loadRecords();
    // Menambahkan event submit pada form agar disimpan
    lewat AJAX.
    document.getElementById('postForm').addEventListener(
    'submit', savePost);
});

// Fungsi untuk menampilkan pesan notifikasi sukses
atau error.
function showAlert(message, type) {
    // Mengambil elemen alert dari halaman.
    const alert = document.getElementById('alert');
    // Mengatur class alert sesuai tipe pesan.
    alert.className = 'alert alert-' + type;
    // Mengisi teks pesan ke dalam alert.
    alert.innerHTML = message;
    // Menampilkan alert.
    alert.style.display = 'block';
    // Menyembunyikan alert otomatis setelah 3 detik.
    setTimeout(() => alert.style.display = 'none',
3000);
}

// Fungsi AJAX untuk mengambil semua data post dari
server.
function loadRecords() {
    // Menampilkan indikator loading.
    document.getElementById('loading').style.display =
'block';
    // Mengirim request ke endpoint crudjs/get_all.
    fetch(baseUrl + '/get_all')
        // Mengubah response server menjadi JSON.
        .then(response => response.json())
        // Memproses data JSON yang diterima.
        .then(result => {
            // Menyembunyikan indikator loading setelah
response diterima.

```

```

        document.getElementById('loading').style.display = 'none';
        // Mengambil elemen tbody tabel.
        const tbody =
document.getElementById('postsBody');
        // Mengosongkan isi tabel sebelum diisi
data terbaru.
        tbody.innerHTML = '';

        // Mengecek apakah response sukses dan
memiliki data.
        if (result.status === 'success' &&
result.data.length > 0) {
            // Melakukan perulangan untuk setiap
data post.
            result.data.forEach(row => {
                // Menambahkan baris tabel baru
untuk setiap post.
                tbody.innerHTML += `
                    <tr>
                        <td>${row.id}</td>
                        <td>${row.image_url ? `` : `'-
'}</td>
                        <td>${escapeHtml(row.title
|| '')}</td>
                        <td>${escapeHtml(row.author
|| '')}</td>
                        <td>${escapeHtml((row.artic
le || '').substring(0, 100))}</td>
                        <td>${row.created_at || `'-
'}</td>
                        <td class="actions">
                            <button class="btn btn-
warning" onclick="editPost(${row.id})">Edit</button>
                            <button class="btn btn-
danger" onclick="deletePost(${row.id})">Delete</button>
                        </td>

```

```

        </tr>
        `;
    });
    // Jika data kosong atau response bukan
    success.
    } else {
        // Menampilkan baris keterangan bahwa
        belum ada data.
        tbody.innerHTML = '<tr><td colspan="7"
style="text-align:center;">Belum ada data</td></tr>';
    }
})
// Menangkap error jika request AJAX gagal.
.catch(error => {
    // Menyembunyikan indikator loading.
    document.getElementById('loading').style.di
splay = 'none';
    // Menampilkan pesan error ke user.
    showAlert('Error loading records: ' +
error.message, 'error');
});
}

// Fungsi untuk membuka modal tambah data.
function openModal() {
    // Mengatur judul modal menjadi Tambah Post.
    document.getElementById('modalTitle').innerText =
'Tambah Post';
    // Mengosongkan semua input form.
    document.getElementById('postForm').reset();
    // Mengosongkan ID agar form dianggap sebagai
    tambah data.
    document.getElementById('postId').value = '';
    // Mengosongkan preview gambar lama.
    document.getElementById('currentImage').innerHTML =
'';
    // Menampilkan modal ke layar.

```

```

        document.getElementById('postModal').style.display
= 'block';
    }

// Fungsi untuk menutup modal.
function closeModal() {
    // Menyembunyikan modal dari layar.
    document.getElementById('postModal').style.display
= 'none';
}

// Fungsi untuk menyimpan data tambah atau edit
menggunakan AJAX.
function savePost(event) {
    // Mencegah form reload halaman secara default.
    event.preventDefault();
    // Mengambil nilai ID untuk menentukan tambah atau
update.
    const id = document.getElementById('postId').value;
    // Mengambil semua data form termasuk file gambar.
    const formData = new
FormData(document.getElementById('postForm'));
    // Menentukan URL tujuan: update jika ada ID, store
jika tidak ada ID.
    const url = id ? baseUrl + '/update/' + id :
baseUrl + '/store';

    // Mengirim data form ke server menggunakan method
POST.
    fetch(url, { method: 'POST', body: formData })
        // Mengubah response server menjadi JSON.
        .then(response => response.json())
        // Memproses hasil simpan/update dari server.
        .then(result => {
            // Mengecek apakah proses simpan/update
berhasil.
            if (result.status === 'success') {
                // Menutup modal setelah sukses.

```

```

        closeModal();
        // Memuat ulang data tabel agar data
terbaru tampil.
        loadRecords();
        // Menampilkan pesan sukses.
        showAlert(result.message, 'success');
        // Jika server mengirim status error.
    } else {
        // Menampilkan pesan error dari server.
        showAlert(result.message, 'error');
    }
})
// Menangkap error jika request AJAX gagal.
.catch(error => showAlert('Error saving record:
' + error.message, 'error'));
}

// Fungsi untuk mengambil data yang akan diedit
berdasarkan ID.
function editPost(id) {
    // Mengirim request ke endpoint get_record dengan
ID post.
    fetch(baseUrl + '/get_record/' + id)
        // Mengubah response server menjadi JSON.
        .then(response => response.json())
        // Memproses data post yang diterima.
        .then(result => {
            // Mengecek apakah data berhasil ditemukan.
            if (result.status === 'success') {
                // Menyimpan data post ke variabel row.
                const row = result.data;
                // Mengubah judul modal menjadi Edit
Post.
                document.getElementById('modalTitle').i
nnerText = 'Edit Post';
                // Mengisi input hidden ID dengan ID
post.

```



```

        document.getElementById('postId').value
= row.id;
        // Mengisi input title dengan data dari
server.
        document.getElementById('title').value
= row.title || '';
        // Mengisi input author dengan data
dari server.
        document.getElementById('author').value
= row.author || '';
        // Mengisi textarea article dengan data
dari server.
        document.getElementById('article').valu
e = row.article || '';
        // Menampilkan preview gambar jika post
memiliki gambar.
        document.getElementById('currentImage')
.innerHTML = row.image_url ? `` : '';
        // Menampilkan modal edit.
        document.getElementById('postModal').st
yle.display = 'block';
        // Jika data gagal ditemukan.
    } else {
        // Menampilkan pesan error dari server.
        showAlert(result.message, 'error');
    }
});
}

// Fungsi untuk menghapus post berdasarkan ID.
function deletePost(id) {
    // Menampilkan konfirmasi sebelum data dihapus.
    if (!confirm('Yakin ingin menghapus data ini?'))
return;

    // Mengirim request hapus ke endpoint delete dengan
method POST.

```

```

    fetch(baseUrl + '/delete/' + id, { method: 'POST'
  })
    // Mengubah response server menjadi JSON.
    .then(response => response.json())
    // Memproses hasil hapus dari server.
    .then(result => {
      // Mengecek apakah data berhasil dihapus.
      if (result.status === 'success') {
        // Memuat ulang data tabel setelah
hapus.

        loadRecords();
        // Menampilkan pesan sukses.
        showAlert(result.message, 'success');
        // Jika hapus gagal.
      } else {
        // Menampilkan pesan error dari server.
        showAlert(result.message, 'error');
      }
    })
    // Menangkap error jika request AJAX gagal.
    .catch(error => showAlert('Error deleting
record: ' + error.message, 'error'));
}

// Fungsi untuk mengamankan teks agar tidak menjalankan
HTML/script berbahaya.
function escapeHtml(text) {
  // Membuat elemen div sementara.
  const div = document.createElement('div');
  // Memasukkan teks sebagai textContent agar
karakter HTML di-escape.
  div.textContent = text;
  // Mengembalikan teks yang sudah aman dalam bentuk
HTML.
  return div.innerHTML;
}

```

```
// Event global untuk menutup modal saat area luar
modal diklik.
window.onclick = function (event) {
    // Mengambil elemen modal.
    const modal = document.getElementById('postModal');
    // Mengecek apakah area yang diklik adalah
background modal.
    if (event.target === modal) {
        // Menutup modal jika user klik area luar
konten modal.
        closeModal();
    }
};
</script>
</body>
</html>
```

Header

```
<!DOCTYPE html>
<!-- Menandai awal dokumen HTML. -->
<html lang="en">
<head>
    <!-- Mengatur encoding karakter agar teks tampil
benar. -->
    <meta charset="UTF-8">
    <!-- Mengatur tampilan agar responsif di perangkat
mobile. -->
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <!-- Menampilkan judul halaman dari controller,
jika tidak ada memakai default CRUD with AJAX. -->
    <title><?php echo isset($title) ? $title : 'CRUD
with AJAX'; ?> - CI3 CRUD</title>
    <!-- CSS internal untuk styling halaman CRUD AJAX.
-->
    <style>
```

```
/* Reset margin, padding, dan box sizing semua
elemen. */
* { margin: 0; padding: 0; box-sizing: border-
box; }

/* Styling dasar body halaman. */
body { font-family: 'Segoe UI', Tahoma, Geneva,
Verdana, sans-serif; background-color: #f5f5f5; line-
height: 1.6; }

/* Container untuk membatasi lebar konten. */
.container { max-width: 1200px; margin: 0 auto;
padding: 20px; }

/* Header halaman dengan background gradasi. */
header { background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%); color: white; padding: 30px
0; margin-bottom: 30px; box-shadow: 0 4px 6px
rgba(0,0,0,.1); }

/* Styling judul utama di header. */
header h1 { text-align: center; font-size:
2.5em; margin-bottom: 10px; }

/* Styling deskripsi singkat di header. */
header p { text-align: center; opacity: .9; }

/* Card pembungkus konten utama. */
.card { background: white; border-radius: 8px;
padding: 25px; margin-bottom: 20px; box-shadow: 0 2px
4px rgba(0,0,0,.1); }

/* Styling dasar semua tombol. */
.btn { padding: 10px 20px; border: none;
border-radius: 4px; cursor: pointer; font-size: 14px;
text-decoration: none; display: inline-block; margin:
2px; transition: all .3s ease; }

/* Warna tombol utama. */
.btn-primary { background: #667eea; color:
white; }

/* Warna tombol sukses/simpan. */
.btn-success { background: #28a745; color:
white; }

/* Warna tombol peringatan/edit. */
```

```
.btn-warning { background: #ffc107; color:
#212529; }
/* Warna tombol bahaya/delete. */
.btn-danger { background: #dc3545; color:
white; }
/* Warna tombol sekunder/batal. */
.btn-secondary { background: #6c757d; color:
white; }
/* Efek saat tombol diarahkan kursor. */
.btn:hover { opacity: .85; transform:
translateY(-1px); }
/* Jarak bawah setiap grup form. */
.form-group { margin-bottom: 15px; }
/* Styling label form. */
.form-group label { display: block; margin-
bottom: 5px; font-weight: bold; color: #333; }
/* Styling input dan textarea form. */
.form-control { width: 100%; padding: 10px;
border: 1px solid #ddd; border-radius: 4px; font-size:
14px; }
/* Styling khusus textarea artikel. */
textarea.form-control { min-height: 120px;
resize: vertical; }
/* Styling dasar tabel data. */
.table { width: 100%; border-collapse:
collapse; margin-top: 20px; }
/* Styling sel header dan isi tabel. */
.table th, .table td { padding: 12px; text-
align: left; border-bottom: 1px solid #ddd; vertical-
align: top; }
/* Warna header tabel. */
.table th { background-color: #667eea; color:
white; }
/* Efek hover pada baris tabel. */
.table tr:hover { background-color: #f8f9fa; }
/* Ukuran preview gambar di tabel dan modal. */
.image-preview { max-width: 100px; max-height:
100px; border-radius: 4px; object-fit: cover; }
```

```

    /* Styling dasar kotak alert. */
    .alert { padding: 12px 15px; border-radius:
4px; margin-bottom: 15px; display: none; }
    /* Styling alert sukses. */
    .alert-success { background: #d4edda; color:
#155724; border: 1px solid #c3e6cb; }
    /* Styling alert error. */
    .alert-error { background: #f8d7da; color:
#721c24; border: 1px solid #f5c6cb; }
    /* Styling background modal. */
    .modal { display: none; position: fixed; z-
index: 1000; left: 0; top: 0; width: 100%; height:
100%; background: rgba(0,0,0,.5); }
    /* Styling kotak isi modal. */
    .modal-content { background: white; margin: 5%
auto; padding: 25px; border-radius: 8px; width: 90%;
max-width: 600px; max-height: 90vh; overflow-y: auto; }
    /* Styling tombol close pada modal. */
    .close { float: right; font-size: 28px; font-
weight: bold; cursor: pointer; color: #aaa; }
    /* Warna tombol close saat hover. */
    .close:hover { color: #000; }
    /* Styling teks loading. */
    .loading { text-align: center; padding: 20px;
color: #666; }
    /* Membuat tombol aksi edit/delete tetap satu
baris. */
    .actions { white-space: nowrap; }
    /* Aturan responsif untuk layar kecil. */
    @media (max-width: 768px) {
        /* Mengecilkan ukuran font tabel pada
mobile. */
        .table { font-size: 12px; }
        /* Mengecilkan padding sel tabel pada
mobile. */
        .table th, .table td { padding: 8px; }
        /* Mengecilkan judul header pada mobile. */
        header h1 { font-size: 2em; }

```

```

        /* Menyesuaikan lebar modal pada mobile. */
        .modal-content { margin: 10% auto; width:
95%; }
    }
</style>
</head>
<body>
    <!-- Header halaman CRUD with AJAX. -->
    <header>
        <!-- Container agar isi header sejajar dengan
konten utama. -->
        <div class="container">
            <!-- Judul utama halaman. -->
            <h1>CRUD with AJAX</h1>
            <!-- Deskripsi singkat teknologi yang
digunakan. -->
            <p>CodeIgniter 3 + HMVC + AJAX tanpa
refresh halaman</p>
        </div>
    </header>

```

Index

```

<?php $this->load->view('crudjs/header'); // Memuat
bagian header, CSS, dan judul halaman CRUD AJAX. ?>

<!-- Container utama untuk isi halaman CRUD AJAX. -->
<div class="container">
    <!-- Tempat menampilkan pesan sukses atau error
dari proses AJAX. -->
    <div id="alert" class="alert"></div>

    <!-- Card pembungkus tombol aksi dan tabel data
post. -->
    <div class="card">
        <!-- Tombol untuk membuka modal tambah post. --
>

```

```

        <button class="btn btn-primary"
onclick="openModal()">Tambah Post</button>
        <!-- Tombol untuk memuat ulang data dari server
menggunakan AJAX. -->
        <button class="btn btn-secondary"
onclick="loadRecords()">Refresh Data</button>

        <!-- Indikator loading saat data sedang diambil
dari server. -->
        <div id="loading" class="loading"
style="display:none;">Loading...</div>

        <!-- Tabel untuk menampilkan data post hasil
response AJAX. -->
        <table class="table" id="postsTable">
            <!-- Bagian kepala tabel. -->
            <thead>
                <!-- Baris judul kolom tabel. -->
                <tr>
                    <!-- Kolom ID post. -->
                    <th>ID</th>
                    <!-- Kolom gambar post. -->
                    <th>Image</th>
                    <!-- Kolom judul post. -->
                    <th>Title</th>
                    <!-- Kolom penulis post. -->
                    <th>Author</th>
                    <!-- Kolom isi artikel post. -->
                    <th>Article</th>
                    <!-- Kolom tanggal dibuat. -->
                    <th>Created</th>
                    <!-- Kolom tombol aksi edit dan
delete. -->
                    <th>Actions</th>
                </tr>
            </thead>
            <!-- Bagian body tabel akan diisi otomatis
oleh JavaScript AJAX. -->

```



```

        <tbody id="postsBody"></tbody>
    </table>
</div>
</div>

<!-- Modal/form pop-up untuk tambah dan edit post. -->
<div id="postModal" class="modal">
    <!-- Kotak isi modal. -->
    <div class="modal-content">
        <!-- Tombol untuk menutup modal. -->
        <span class="close"
onclick="closeModal()">&times;</span>
        <!-- Judul modal yang berubah antara Tambah
Post dan Edit Post. -->
        <h2 id="modalTitle">Tambah Post</h2>
        <!-- Form post dengan enctype multipart agar
bisa upload gambar. -->
        <form id="postForm" enctype="multipart/form-
data">
            <!-- Input hidden untuk menyimpan ID saat
proses edit. -->
            <input type="hidden" id="postId" name="id">
            <!-- Grup input judul post. -->
            <div class="form-group">
                <!-- Label input title. -->
                <label for="title">Title</label>
                <!-- Input teks untuk judul post. -->
                <input type="text" id="title"
name="title" class="form-control" required>
            </div>
            <!-- Grup input author. -->
            <div class="form-group">
                <!-- Label input author. -->
                <label for="author">Author</label>
                <!-- Input teks untuk nama penulis
post. -->
                <input type="text" id="author"
name="author" class="form-control" required>

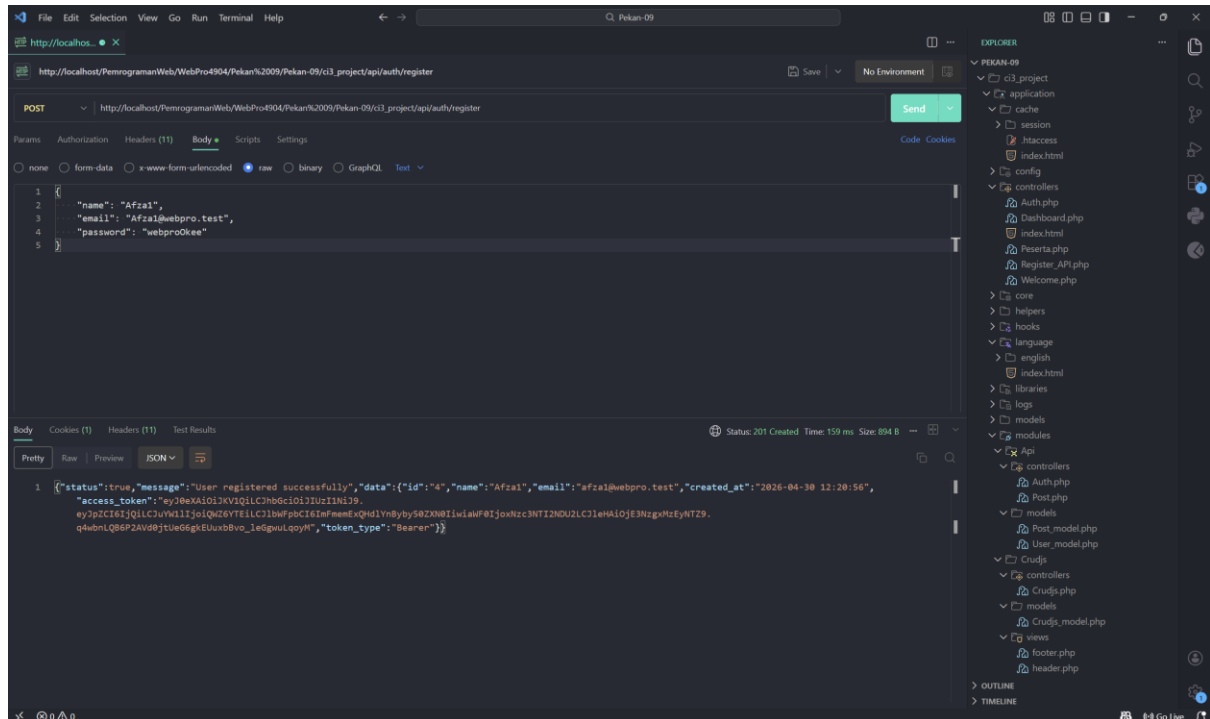
```

```

        </div>
        <!-- Grup input article. -->
        <div class="form-group">
            <!-- Label textarea article. -->
            <label for="article">Article</label>
            <!-- Textarea untuk isi artikel post. -
->
                <textarea id="article" name="article"
class="form-control" required></textarea>
        </div>
        <!-- Grup input image. -->
        <div class="form-group">
            <!-- Label input image. -->
            <label for="image">Image</label>
            <!-- Input file untuk memilih gambar
dari komputer. -->
                <input type="file" id="image"
name="image" class="form-control" accept="image/*">
            <!-- Tempat menampilkan gambar saat
sedang edit post. -->
                <div id="currentImage" style="margin-
top:10px;"></div>
        </div>
        <!-- Tombol submit untuk menyimpan
tambah/edit data. -->
            <button type="submit" class="btn btn-
success">Simpan</button>
            <!-- Tombol untuk membatalkan dan menutup
modal. -->
                <button type="button" class="btn btn-
secondary" onclick="closeModal()">Batal</button>
        </form>
    </div>
</div>

<?php $this->load->view('crudjs/footer'); // Memuat
JavaScript AJAX dan penutup HTML. ?>

```



Analisis dan Pembahasan

1. Analisis Arsitektur Sistem

Aplikasi yang dibuat ini pakai arsitektur Client-Server yang sangat mendasar. PHP di sini tugasnya jadi "otak" di sisi server (*back-end*), sementara MySQL jadi "gudang" penyimpanannya (*database*).

Poin penting yang saya amati adalah penggunaan file config.php sebagai pusat koneksi. Ini efisien banget, karena kalau nanti ada perubahan **username** atau **password** database, kita nggak perlu repot-repot buka semua file satu per satu, cukup ganti di satu file itu saja.

2. Di bagian registrasi, hal yang paling krusial adalah fungsi password_hash(). Pas saya coba daftar, password yang masuk ke database itu bentuknya jadi kode acak yang panjang banget. Ini bagus untuk keamanan; jadi walaupun database-nya diintip orang, password asli user nggak bakal ketahuan.

3. Bagian paling menarik ada di halaman utama (index.php). Di sini, satu form bisa punya dua nyawa: buat input data baru atau buat update data lama.

Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa:

- Bagaimana **MVC memisahkan tanggung jawab** sehingga kode lebih mudah dipelihara.
- Pentingnya **validasi berlapis** (client-side dan server-side).
- Cara kerja **session management** dalam aplikasi web.
- Penanganan **file upload** secara aman di server.
- Perbedaan antara **web endpoint** (menghasilkan HTML) dan **API endpoint** (menghasilkan JSON).